DISTRIBUTED JOB QUEUE & TASK PROCESSING SYSTEM

Comprehensive Project Master Documentation & Viva Examination Guide
200 In-Depth Technical Viva Questions & Answers (Q1 – Q200)

Author / Candidate: Koushik Bobba

Target Role: SDE / Backend Developer

Repository: github.com/koushikbobba/distributed-job-queue

Tech Stack: Python 3.11, Django, DRF, Celery, Redis, PostgreSQL

Infrastructure: Docker, Prometheus, Grafana, AWS ECS/RDS

Verification Status: 18/18 Automated Tests Passing

PART 1: COMPREHENSIVE PROJECT DOCUMENTATION

1. Executive Summary & Purpose

In modern web applications, performing long-running computations (e.g., generating annual financial reports, batch email delivery, data aggregation, or remote API synchronization) inside synchronous HTTP request-response handlers causes severe production failures: HTTP 504 gateway timeouts, worker thread exhaustion, and vulnerability to duplicate executions upon network retries. This project implements a production-style, fault-tolerant **Distributed Job Queue & Task Processing Platform**. It decouples HTTP request ingestion from background processing, achieving sub-10ms API responses while processing asynchronous tasks with strict multi-priority scheduling, distributed locking, atomic state transitions, at-least-once delivery guarantees, exponential backoff with full jitter, and full observability via Prometheus and Grafana.

2. System Architecture & Component Interaction

The system consists of five distinct architectural tiers:

- **API & Security Tier (Django REST Framework):** Ingests jobs via REST API, validates JWT authentication, rotates tokens, blacklists tokens upon logout, enforces sliding-window rate limits (100 req/min general, 10 job submissions/min), and calculates SHA-256 request payload hashes for idempotency.
- **Persistent Storage Tier (PostgreSQL 15):** Acts as the ACID source of truth. Stores custom user models (UUID PKs), macro job state, 1:1 result outputs (`JobResult`), 1:N immutable worker attempt history (`JobExecution`), and idempotency records with composite B-tree indexes.
- **Message Broker & Coordination Tier (Redis 7.0):** Manages multi-priority FIFO/ZSET message queues (`high_priority`, `default_priority`, `low_priority`), distributed locks (`SETNX` with TTL), and rate limiting buckets.

• **Asynchronous Execution Tier (Celery 5.3+):** Distributed worker processes executing a pluggable handler registry (`BaseJobHandler`), enforcing prefetch control (`prefetch_multiplier=1`), at-least-once delivery (`acks_late=True`), safe Lua lock release, and exponential backoff retries with full jitter.

• **Telemetry & Observability Tier (Prometheus & Grafana):** Scrapes `/metrics` every 5 seconds, recording execution duration histograms (p95/p99 latency), throughput counters by status, active worker gauges, and retry frequencies.

3. Core Distributed Systems Engineering Solutions

A. Dual-Write Post-Commit Safety (`transaction.on_commit`): Publishing a Celery task inside an uncommitted database transaction creates a race condition where a worker pops the message from Redis before PostgreSQL has committed the record, causing a `Job.DoesNotExist` crash. The system registers `transaction.on_commit(lambda: task.apply_async(...))`, guaranteeing messages reach Redis only after PostgreSQL acknowledges the commit.

B. Redis Distributed Locking with Atomic Lua Script: Workers acquire locks using `SET lock:job: NX PX 120000`. To prevent a slow worker from deleting a subsequent worker's lock after TTL expiration, release is executed via an atomic Lua script: if redis.call('get', KEYS[1]) == ARGV[1] then return redis.call('del', KEYS[1]) else return 0 end.

C. Idempotency & Tampering Detection: The server hashes SHA256(user_id + endpoint + payload). If the key exists with a matching hash, it returns the cached HTTP 202 response (`X-Cache-Lookup: HIT-Idempotency`). If the hash differs, it rejects with HTTP 422 (Tampered Payload). Concurrent identical requests are intercepted by PostgreSQL's compound unique constraint.

D. Exponential Backoff with Full Random Jitter: To eliminate the Thundering Herd problem on downstream services, retry countdowns are randomized: delay = random.uniform(0, min(300, 2^retry_count * base_delay)).

PART 2: MASTER VIVA & TECHNICAL INTERVIEW QUESTIONS (Q1 – Q200)

Exhaustive continuous sequence of technical examination questions and comprehensive model answers.

Q1. What is the primary objective of this project?

Answer: The primary objective is to build a production-grade, fault-tolerant distributed job queue and task processing platform that decouples synchronous HTTP request-response lifecycles from long-running, compute-heavy, or I/O-bound background tasks, ensuring sub-10ms API responses and reliable asynchronous execution.

Q2. What are some concrete examples of background tasks handled by this system?

Answer: Sending bulk/transactional emails (send_email), generating heavy PDF/Excel financial reports (generate_report), processing and transforming batches of data (process_data), and simulating long-running asynchronous workflows (simulate_long_task).

Q3. Why can't long-running tasks be executed directly inside standard Django views?

Answer: Executing long tasks inside Django views blocks WSGI/ASGI worker threads for the task duration, leading to HTTP 504 Gateway Timeouts, web server thread exhaustion, and complete denial of service for concurrent users.

Q4. What is the complete technology stack used across the project?

Answer: Python 3.11, Django 4.2+, Django REST Framework, Celery 5.3+, Redis 7.0, PostgreSQL 15, Docker & Docker Compose, Prometheus, Grafana, and Pytest.

Q5. What exact role does PostgreSQL play in this architecture?

Answer: PostgreSQL serves as the persistent, ACID-compliant source of truth. It stores user accounts, macro job state machines, 1:N execution audit logs (JobExecution), 1:1 output payloads (JobResult), and idempotency keys with composite B-tree indexes.

Q6. Why do we need Redis if PostgreSQL is already present?

Answer: Redis provides three specialized in-memory capabilities: (1) an ultra-low-latency Celery message broker supporting non-blocking priority pops (BRPOP), (2) a high-speed distributed locking engine (SETNX), and (3) a sliding-window rate-limiting cache.

Q7. Why not store all job states and queues exclusively in Redis?

Answer: Redis lacks relational integrity, complex composite B-tree indexing, and long-term ACID transactional durability needed for user dashboards, historical auditing, compliance, and large payload queries.

Q8. Explain the role of Celery in this architecture.

Answer: Celery runs as an independent distributed worker pool, polling Redis queues in strict priority order, managing worker concurrency slots, executing task logic, handling retries, and capturing worker crash signals.

Q9. Why did you choose UUIDv4 for model primary keys instead of auto-incrementing integers?

Answer: UUIDv4 prevents ID enumeration attacks in multi-tenant REST APIs (where attackers guess /api/jobs/1, /api/jobs/2), eliminates collision risks across distributed database shards, and allows IDs to be safely generated before DB insertion.

Q10. Can using UUIDv4 as primary keys cause database performance degradation in PostgreSQL?

Answer: Yes, because random UUIDv4 values cause B-tree leaf page splits and index fragmentation compared to sequential integers. However, for our security and multi-tenant isolation requirements, UUIDv4 is optimal. UUIDv7 can be used in future iterations for time-ordered locality.

Q11. Explain the difference between the Job and JobExecution models.

Answer: Job represents the aggregate entity and its current macro state (e.g., COMPLETED, RETRYING). JobExecution is an immutable 1-to-N audit log that records every single worker execution attempt with timestamps, worker IDs, duration, and tracebacks.

Q12. Why did you separate JobResult into its own table instead of adding a JSONB column inside Job?

Answer: JobResult stores output payloads of completed tasks, which can be large. By isolating it in a 1:1 table, querying or listing jobs (GET /api/jobs/) only fetches lightweight Job metadata, minimizing disk I/O and memory overhead.

Q13. What are all the states in your job lifecycle state machine?

Answer: QUEUED, PROCESSING, COMPLETED, FAILED, RETRYING, and CANCELLED.

Q14. Which states in your state machine are terminal?

Answer: COMPLETED, FAILED, and CANCELLED. Once reached, no further transitions or execution attempts are permitted.

Q15. Explain how a job transitions from QUEUED to PROCESSING.

Answer: When a Celery worker pops a task from Redis, it checks if the job was cancelled. If not, it acquires a Redis distributed lock, opens an ACID transaction on PostgreSQL, sets status=PROCESSING, stamps started_at, records worker_id, and creates a JobExecution record.

Q16. What happens if a job fails during execution? What state does it enter?

Answer: If an exception occurs, the worker increments current_retry_count and records the failure in JobExecution. If current_retry_count <= max_retries, Job transitions to RETRYING and is re-queued with exponential backoff delay. If retries are exhausted, it transitions to FAILED.

Q17. What is the Dual-Write problem in the context of job creation?

Answer: When creating a job, the system must write to PostgreSQL (saving the Job row) and Redis (publishing the task message). If one write succeeds and the other fails, or if timing is uncoordinated, data inconsistency occurs.

Q18. What specific race condition occurs if you call task.apply_async() inside an open Django database transaction?

Answer: Celery task dispatch over Redis takes < 1ms. If dispatched before the DB transaction commits, a worker pops the message and queries PostgreSQL (Job.objects.get(id=job_id)) before the web server has committed the row, crashing with Job.DoesNotExist.

Q19. How did you resolve this post-commit race condition in your code?

Answer: By wrapping the Celery task dispatch in Django's transaction.on_commit() hook: with transaction.atomic(): job = serializer.save(); transaction.on_commit(lambda: execute_job_task.apply_async(args=[str(job.id)], queue=queue_name)).

Q20. What happens if the database transaction fails or rolls back? Does the Celery task get dispatched?

Answer: No. Callbacks registered with transaction.on_commit() are discarded if the transaction rolls back, preventing phantom tasks from being dispatched to Celery.

Q21. What happens if the database commit succeeds, but Redis is down when transaction.on_commit() executes?

Answer: The on_commit callback raises an exception during task publishing. The HTTP request returns an error to the client, but the job remains in PostgreSQL as QUEUED. A periodic sweeper task queries for stale QUEUED jobs and re-publishes them.

Q22. What is Idempotency and why is it critical in distributed task queues?

Answer: Idempotency ensures that making the same API request multiple times produces the exact same outcome as making it once. It prevents duplicate job creation when network retries occur.

Q23. How does your system implement API idempotency?

Answer: The client sends an Idempotency-Key header. The server computes a SHA-256 hash of (user_id + request_path + request_body) and queries IdempotencyKey. If found with a cached response, it returns the cached HTTP 202. If not, it reserves the key in DB and creates the job.

Q24. What happens if an attacker sends an existing Idempotency-Key with a modified payload?

Answer: The server detects payload tampering by comparing the computed SHA-256 hash with the stored request_hash. If they do not match, it rejects the request with HTTP 422 Unprocessable Entity (code: idempotency_payload_mismatch).

Q25. What happens if two identical requests with the same Idempotency-Key arrive at the exact same millisecond on two different web pods?

Answer: Both pods attempt to insert into idempotency_keys. PostgreSQL's UniqueConstraint(fields=['user', 'idempotency_key']) allows only one insert. The second pod catches IntegrityError and returns HTTP 409 Conflict.

Q26. Why do idempotency keys have an expiration time (expires_at)?

Answer: Storing idempotency keys indefinitely causes unbounded database table growth. Keys expire after 24 hours, after which clients must generate a new key for subsequent operations.

Q27. How are expired idempotency keys cleaned up?

Answer: The expires_at column is indexed (idx_idemp_expires_at). A periodic background maintenance task deletes records where expires_at <= timezone.now().

Q28. What is a Distributed Lock, and why is it needed in this project?

Answer: A distributed lock is a synchronization mechanism guaranteeing mutual exclusion across independent processes/nodes, ensuring that duplicate message deliveries never result in concurrent execution of the same job.

Q29. How is the distributed lock acquired in Redis?

Answer: Using the atomic command: SET lock:job:<job_id> <unique_token> NX PX <ttl_ms>. NX ensures the key is set only if it does not exist; PX sets an automatic expiration TTL in milliseconds.

Q30. Why is a TTL (Time-To-Live) mandatory when acquiring a distributed lock?

Answer: If a worker acquires a lock without a TTL and immediately crashes, suffers an OOM kill, or loses power, the lock would remain in Redis forever, causing a permanent deadlock.

Q31. Explain the Lock Release Race Condition and how you solved it.

Answer: If worker A's lock expires due to a long GC pause, worker B acquires the lock. If worker A resumes and blindly runs DEL, it deletes worker B's active lock! We solved this by generating a unique UUID token per acquisition and releasing via an atomic Lua script that checks token ownership before deleting.

Q32. Why must token verification and deletion be executed as an atomic Lua script rather than two separate Redis commands?

Answer: If executed as separate GET and DEL commands, a context switch could occur between them during which the lock expires and is acquired by another worker. Lua scripts execute atomically inside Redis.

Q33. What is the Thundering Herd problem in task retries?

Answer: If a shared service fails, hundreds of tasks fail simultaneously. If they all retry at exact fixed exponential intervals (2s, 4s, 8s), they will hit the recovering service at the exact same second, crashing it again.

Q34. How did you implement Exponential Backoff with Full Jitter?

Answer: Using: delay = random.uniform(0, min(300, 2^retry_count * base_delay)). Randomizing the delay between 0 and the exponential maximum spreads retry traffic uniformly across the timeline.

Q35. What is the difference between Full Jitter, Equal Jitter, and Decorrelated Jitter?

Answer: Full Jitter chooses a uniform random value between 0 and 2^n * base. Equal Jitter keeps half the backoff fixed and randomizes the rest. Decorrelated Jitter calculates delay based on previous sleep. Full Jitter provides the best collision reduction for network requests.

Q36. Explain the Pluggable Task Registry Pattern in apps/workers/registry.py.

Answer: Instead of a monolithic if/elif block checking job types, we created an abstract BaseJobHandler with an execute(job, payload) method. Concrete classes implement this interface and register with JobRegistry, adhering to the Open/Closed Principle.

Q37. What happens if a job is submitted with an unregistered job_type?

Answer: The serializer validates job_type against JobRegistry.get_supported_types() during request validation, rejecting invalid types with HTTP 400 Bad Request before the job is ever created or queued.

Q38. How does Celery implement priority queues with Redis?

Answer: Celery creates multiple Redis lists (high_priority, default_priority, low_priority). Workers are configured to inspect queues in priority order: -Q high_priority,default_priority,low_priority.

Q39. How does the Celery worker ensure high-priority tasks are processed before low-priority tasks?

Answer: The worker issues a blocking pop (BRPOP queue_high queue_default queue_low 0). Redis checks the lists from left to right, always popping elements from queue_high first if any exist.

Q40. What is worker_prefetch_multiplier and why did you set it to 1?

Answer: By default, Celery prefetches 4 * concurrency tasks from Redis into local worker memory. If a worker prefetches multiple heavy tasks, subsequent fast tasks are blocked in memory while other idle workers have empty queues. Setting prefetch=1 ensures fair 1-by-1 task dispatch.

Q41. What is the difference between acks_late = False and acks_late = True in Celery?

Answer: With acks_late=False (default), Celery acknowledges the message immediately upon receiving it before execution starts. If the worker crashes, the message is lost. With acks_late=True, the worker acknowledges only after execution completes.

Q42. What does task_reject_on_worker_lost = True do?

Answer: When combined with acks_late=True, if a worker process is terminated by the OS (SIGKILL, OOM killer), the Celery master process rejects the message and Redis re-delivers it to another available worker.

Q43. Because of acks_late = True, what message delivery guarantee does this system provide?

Answer: At-Least-Once Delivery. Tasks are guaranteed never to be lost, but under worker crash scenarios, they may be delivered more than once. This makes idempotency and distributed locking mandatory.

Q44. How does Job Cancellation work in this system?

Answer: When POST /api/jobs/{id}/cancel/ is called: If QUEUED, status is set to CANCELLED in DB and revoked in Celery. If PROCESSING, status is set to CANCELLED in DB and non-terminating revocation is broadcast. If terminal, returns HTTP 400.

Q45. Why did you set terminate=False instead of terminate=True during Celery task revocation?

Answer: terminate=True forcefully kills worker processes with SIGKILL mid-execution, risking corrupted DB transactions, unreleased locks, and resource leaks. We use cooperative cancellation where handlers inspect status at safe boundaries.

Q46. How does your API Rate Limiting work?

Answer: Rate limiting is implemented using DRF throttles backed by Redis: AuthenticatedUserRateThrottle (100 req/min general) and JobSubmissionRateThrottle (10 submissions/min). Exceeding limits returns HTTP 429 Too Many Requests.

Q47. How does JWT authentication work in this project?

Answer: Using djangorestframework-simplejwt, users log in with email/password and receive an Access Token (60 min) and Refresh Token (7 days). The access token is sent as a Bearer header on protected endpoints.

Q48. How is user logout and JWT revocation handled?

Answer: Since JWTs are stateless, logging out requires blacklisting the refresh token. The client calls POST /api/auth/logout/ with the refresh token, which is inserted into PostgreSQL's token_blacklist table.

Q49. Explain the composite index idx_jobs_user_status_created on (user_id, status, created_at DESC).

Answer: When querying user jobs by status ordered by date, PostgreSQL uses this composite index to satisfy both the filter and sorting predicates in a single Index Scan, eliminating in-memory sorting.

Q50. What would happen if you had 3 separate single-column indexes on user_id, status, and created_at instead of the composite index?

Answer: PostgreSQL would perform a BitmapAnd index scan across separate indexes, fetch heap pages from disk, and execute an expensive in-memory/disk sort for ORDER BY created_at DESC.

Q51. What metrics does the system export to Prometheus?

Answer: job_queue_jobs_total (Counter), job_queue_execution_duration_seconds (Histogram), job_queue_active_jobs (Gauge), job_queue_retries_total (Counter), job_queue_idempotency_cache_hits_total (Counter), and job_queue_rate_limit_exceeded_total (Counter).

Q52. How does Prometheus scrape metrics from the application?

Answer: Prometheus issues periodic HTTP GET requests (every 5s) to http://web:8000/metrics. django-prometheus serializes current metric data into the standard Prometheus exposition format.

Q53. How is Grafana configured in this project?

Answer: Grafana is auto-provisioned using configuration files in docker/grafana/provisioning/, automatically mounting Prometheus as default datasource and loading job_queue_dashboard.json on startup.

Q54. What key visual panels are on the Grafana dashboard?

Answer: Single-stat gauges for Total Jobs, Active In-Flight Workers, Total Retries, and Idempotency Hits; Time-series graph for Job Throughput by Status; and 95th Percentile Execution Duration histogram chart.

Q55. What does the /api/monitoring/health/ endpoint check?

Answer: It performs active connectivity checks on PostgreSQL (SELECT 1;) and Redis (redis_client.ping()), aggregates active job counts across statuses, and returns HTTP 200 OK (or HTTP 503 if unhealthy).

Q56. What Docker services are defined in docker-compose.yml?

Answer: 6 services: postgres (PostgreSQL 15), redis (Redis 7), web (Django API), celery_worker (Celery worker pool), prometheus (Prometheus engine), and grafana (Grafana dashboard).

Q57. Why are container healthchecks defined for PostgreSQL and Redis in docker-compose.yml?

Answer: To prevent startup race conditions. The web and worker services specify depends_on with condition: service_healthy, delaying startup until PostgreSQL and Redis pass socket healthchecks.

Q58. What do the entrypoint scripts docker/entrypoints/api.sh and worker.sh do?

Answer: They execute network socket polling loops (nc -z) to verify dependencies, run database migrations (python manage.py migrate), and start the application/worker processes.

Q59. How is the Django settings architecture structured?

Answer: Using a split settings pattern in config/settings/: base.py (shared settings), local.py (dev overrides/logging), production.py (SSL/HSTS, JSON logging), and test.py (in-memory SQLite/cache for fast testing).

Q60. How did you test the codebase?

Answer: Using pytest and pytest-django. We wrote 18 automated unit and integration tests covering auth, token blacklisting, job creation, multi-tenant isolation, cancellation, idempotency, tampering, worker execution, and retries.

Q61. How does tests/load_testing/locustfile.py perform load testing?

Answer: It simulates realistic user traffic with weighted task distribution: 50% job submissions with idempotency keys, 30% status polling, 20% filtered listing, and 10% duplicate idempotency cache hit tests.

Q62. How would you deploy this project to AWS in production?

Answer: API and workers on AWS ECS Fargate across private subnets; database on AWS RDS PostgreSQL Multi-AZ with RDS Proxy; broker on AWS ElastiCache for Redis Multi-AZ; and traffic routed via AWS Application Load Balancer (ALB).

Q63. How should Celery workers be auto-scaled in production on AWS?

Answer: Workers should auto-scale based on Redis Queue Depth (LLEN across queues) emitted as a custom CloudWatch metric rather than CPU utilization alone, scaling out when queue depth exceeds a target threshold.

Q64. In execute_job_task, why do you pass job_id (a string) rather than the serialized Job model instance?

Answer: Passing model instances into task arguments can result in stale data when the worker executes. Passing primitive job_id ensures the worker fetches the freshest committed state directly from PostgreSQL.

Q65. What if the job is deleted from PostgreSQL between when it was queued and when the worker pops it?

Answer: The database lookup is wrapped in try...except Job.DoesNotExist. If missing, the worker logs an error and returns {'status': 'dropped', 'reason': 'job_not_found'} without crashing.

Q66. What if a user cancels a job while it is waiting in the Redis queue?

Answer: When the worker pops the task, it immediately checks if job.status == JobStatus.CANCELLED. If so, it returns {'status': 'skipped', 'reason': 'job_cancelled'} without executing or acquiring locks.

Q67. In apps/workers/tasks.py, why do you refresh the job instance using job.refresh_from_db() inside transaction.atomic()?

Answer: Between when the worker fetched the job and acquired the Redis lock, another process (like an API cancellation) might have updated the database row. refresh_from_db() ensures the worker inspects the latest committed state.

Q68. What happens if a Celery task crashes due to a MemoryError or Python segfault?

Answer: The worker process dies immediately. Because task_reject_on_worker_lost=True and task_acks_late=True are set, the Celery master rejects the task and Redis re-delivers it to another worker.

Q69. If a task crashed because of a poisonous payload, won't it cause an infinite crash loop across all workers?

Answer: PostgreSQL tracks current_retry_count. When the next worker picks it up, it sees current_retry_count >= max_retries, marks the job FAILED, and drops the task. Broker-level Dead-Letter Queues (DLQ) handle un-tracked segfaults.

Q70. In apps/workers/locking.py, why did you implement RedisDistributedLock as a Context Manager?

Answer: Implementing __enter__ and __exit__ ensures the distributed lock is deterministically released even if unhandled exceptions or runtime errors occur inside the execution block.

Q71. What happens if the Redis server crashes while a worker is holding a lock?

Answer: The worker continues executing. When releasing in finally, Redis operations fail gracefully with a logged warning. When Redis restarts, the in-memory lock will be cleared, preventing permanent deadlocks.

Q72. In apps/jobs/idempotency.py, why do you serialize the JSON payload with json.dumps(data, sort_keys=True) before hashing?

Answer: JSON dictionary keys are unordered. Without sort_keys=True, {'a': 1, 'b': 2} and {'b': 2, 'a': 1} would produce different SHA-256 hashes. sort_keys=True ensures deterministic hashing.

Q73. Why did you include user_id inside the SHA-256 idempotency signature?

Answer: To ensure multi-tenant isolation. If User A and User B coincidentally submit identical payloads with the same idempotency key string, their hashes and database records remain strictly isolated.

Q74. In apps/jobs/views.py, why does JobViewSet override get_queryset instead of using a global filter?

Answer: Overriding get_queryset() enforces row-level security at the ORM level. Regular users only evaluate Job.objects.filter(user=self.request.user), guaranteeing that guessing another user's UUID returns HTTP 404.

Q75. Why does get_queryset use select_related('result') and prefetch_related('executions')?

Answer: To eliminate the N+1 Query Problem. Without them, serializing 20 jobs with results and execution logs would execute 41 SQL queries. With them, Django executes exactly 2 optimized queries.

Q76. What is the HTTP status code returned upon successful job creation and why?

Answer: HTTP 202 Accepted. It is the RFC-compliant status code indicating that the request has been accepted for processing, but asynchronous background execution has not yet completed.

Q77. What HTTP status code is returned when a duplicate idempotency key is submitted with an identical payload?

Answer: HTTP 202 Accepted, accompanied by custom header X-Cache-Lookup: HIT-Idempotency and the original cached response payload.

Q78. In apps/users/permissions.py, why do you have both has_permission and has_object_permission?

Answer: has_permission runs on all requests before view execution (checking authentication). has_object_permission runs when retrieving/updating a specific instance (/api/jobs/{id}/), checking if obj.user == request.user.

Q79. How does SimpleJWT validate an access token on incoming requests?

Answer: It verifies the cryptographic HMAC-SHA256 signature using the server's SECRET_KEY, checks that the exp timestamp is in the future, and decodes the user_id payload to populate request.user.

Q80. Does DRF SimpleJWT query PostgreSQL on every single API request to validate an access token?

Answer: No. JWT verification is stateless in memory. It only queries PostgreSQL if the view explicitly accesses request.user database attributes.

Q81. In apps/users/serializers.py, what does validate_password(attrs['password']) do?

Answer: It runs Django's configured password validators (length, common passwords, numeric checks), rejecting weak passwords during registration.

Q82. In apps/jobs/serializers.py, what validation is performed on max_retries?

Answer: It enforces min_value=0 and max_value=10 with a default of 3, preventing malicious clients from setting unreasonably high retry counts that would overload workers.

Q83. How do you prevent Django from loading full database models into memory during tests?

Answer: In config/settings/test.py, we configure in-memory SQLite (:memory:) and set PASSWORD_HASHERS to MD5PasswordHasher to make user creation tests run 10x faster.

Q84. In config/celery.py, why do you set task_serializer = 'json' and accept_content = ['json']?

Answer: For security. Older Celery setups defaulted to Python's pickle serializer, which allows arbitrary code execution if an attacker injects messages into Redis. JSON serialization is safe.

Q85. What is the difference between Celery's soft_time_limit and time_limit?

Answer: soft_time_limit (28m) raises SoftTimeLimitExceeded inside the task, allowing graceful cleanup. time_limit (30m) forcefully terminates the worker process with SIGKILL.

Q86. How does django-filter handle filtering in apps/jobs/filters.py?

Answer: JobFilter maps URL query parameters (?status=completed&priority=high) to ORM lookups with lookup_expr='iexact' for case-insensitivity and created_after/before for date ranges.

Q87. In docker-compose.yml, what is the purpose of redis-server --appendonly yes --maxmemory 256mb --maxmemory-policy noeviction?

Answer: --appendonly yes enables AOF persistence so queues survive restarts; --maxmemory 256mb prevents memory exhaustion; and --maxmemory-policy noeviction prevents silent dropping of queue messages.

Q88. What is the difference between Gunicorn and Uvicorn in production deployment?

Answer: Gunicorn is a WSGI HTTP process manager for synchronous workers. Uvicorn is an ASGI server for asynchronous Python. In production, Gunicorn acts as process manager running Uvicorn worker classes.

Q89. In apps/monitoring/metrics.py, why is job_queue_execution_duration_seconds a Histogram rather than a Summary?

Answer: Histograms allow Prometheus to calculate percentile aggregations (like p95 and p99 latencies) across multiple worker instances using histogram_quantile(). Summaries cannot be aggregated across instances.

Q90. What PromQL query computes the 95th percentile job execution duration on Grafana?

Answer: histogram_quantile(0.95, sum by (le, job_type) (rate(job_queue_execution_duration_seconds_bucket[5m])))

Q91. What PromQL query computes job throughput in jobs per minute by status?

Answer: sum by (status) (rate(job_queue_jobs_total[1m])) * 60

Q92. What is the purpose of drf-spectacular in this project?

Answer: It inspects viewsets, serializers, and @extend_schema decorators to generate an OpenAPI 3.0 schema (/api/schema/) powering Swagger UI (/api/docs/) and ReDoc (/api/redoc/).

Q93. How are database migrations executed automatically in Docker?

Answer: The API entrypoint script docker/entrypoints/api.sh runs python manage.py migrate --noinput on startup after verifying PostgreSQL availability.

Q94. Why do you use CONN_MAX_AGE = 60 in database settings?

Answer: It maintains persistent database connections for 60 seconds across requests, eliminating TCP connection setup overhead on every request and reducing database CPU usage.

Q95. In apps/workers/locking.py, what happens if connection_alias fails to connect to Redis during testing?

Answer: The constructor catches the exception, sets self.redis_client = None, and acquire()/release() methods return True, allowing unit tests to run without active Redis instances.

Q96. What is the difference between apply_async and delay in Celery?

Answer: delay() only accepts positional and keyword arguments. apply_async() supports execution options like queue, countdown, eta, and expires.

Q97. How do you prevent a slow worker from starving fast tasks in Celery?

Answer: By setting worker_prefetch_multiplier = 1, disabling batching, and routing heavy tasks to dedicated worker queues.

Q98. What exact data structure is used by Redis to store Celery task messages?

Answer: Redis Lists (using LPUSH and BRPOP). Tasks scheduled with a countdown or eta are stored in a Redis Sorted Set (ZSET) ordered by epoch timestamp.

Q99. How does Celery Beat manage scheduled recurring tasks?

Answer: Celery Beat reads schedule definitions, tracks elapsed intervals, and pushes task execution messages into the Redis queue when the scheduled timestamp arrives.

Q100. In tests/test_idempotency.py, how do you verify duplicate submissions don't create extra rows?

Answer: We submit the initial request, check Job.objects.count(), submit the second request with the same Idempotency-Key, and assert Job count remains 1 and response header is HIT-Idempotency.

Q101. In apps/users/models.py, what does UserManager.normalize_email() do?

Answer: It lowercases the domain portion of email addresses (User@EXAMPLE.com -> User@example.com) to ensure consistent uniqueness lookups.

Q102. Why is REQUIRED_FIELDS = [] set in your custom User model?

Answer: REQUIRED_FIELDS defines prompts for createsuperuser aside from USERNAME_FIELD (email) and password. Since email and password are the only required fields, the list is empty.

Q103. In apps/jobs/models.py, why is status defined with models.TextChoices instead of integers?

Answer: TextChoices stores human-readable strings directly in PostgreSQL, making raw SQL queries, logs, and Prometheus metrics self-documenting without lookup tables.

Q104. What is the storage difference between PostgreSQL JSON and JSONB data types?

Answer: JSON stores exact plain text requiring re-parsing on every read. JSONB stores decomposed binary format supporting GIN indexing, fast lookups, and JSON path operators.

Q105. How does Django handle database migrations for composite indexes?

Answer: In 0001_initial.py, Django executes AddIndex specifying models.Index(fields=['user', 'status', '-created_at']), translating to SQL: CREATE INDEX idx_jobs_user_status_created ON jobs (user_id, status, created_at DESC);.

Q106. In apps/workers/registry.py, how does SendEmailHandler simulate task execution?

Answer: It extracts recipient and subject, verifies recipient presence, sleeps 0.5s to simulate SMTP latency, logs the delivery, and returns a dictionary with byte count and status.

Q107. How does GenerateReportHandler simulate heavy report generation?

Answer: It verifies customer_id, simulates aggregation computation via sleep(1.0), and returns a dictionary containing a mock AWS S3 PDF URL and record count.

Q108. How does SimulateLongTaskHandler test retry behavior?

Answer: If payload contains 'simulate_failure': true, it raises a RuntimeError, triggering Celery's retry mechanism and incrementing current_retry_count.

Q109. In apps/workers/tasks.py, how is worker_node_id generated?

Answer: Using f'worker-{socket.gethostname()}-{self.request.id[:8]}', giving each execution attempt a unique traceable identifier across container clusters.

Q110. What happens if a job has max_retries = 0 and its task handler raises an exception?

Answer: The worker increments current_retry_count to 1. Since 1 > max_retries, it immediately marks the job FAILED, stamps failed_at, updates JobExecution, and schedules no retries.

Q111. What is the purpose of JobRegistry.get_supported_types()?

Answer: It dynamically returns all registered string keys in _handlers (['send_email', 'generate_report', ...]), which serializers use to validate incoming job creation requests.

Q112. In apps/jobs/serializers.py, why is JobDetailSerializer separate from JobListSerializer?

Answer: JobListSerializer returns only essential list columns. JobDetailSerializer includes nested result and executions relations, avoiding serialization overhead on list endpoints.

Q113. In apps/jobs/throttling.py, how does UserRateThrottle identify users?

Answer: For authenticated requests, it uses request.user.pk as cache key (throttle_user_<pk>). For unauthenticated requests, it falls back to client IP address.

Q114. What happens when a rate limit is exceeded in DRF?

Answer: The throttle raises exceptions.Throttled(wait), which DRF renders as HTTP 429 Too Many Requests with a Retry-After header.

Q115. In apps/users/views.py, why does RegisterView set permission_classes = [AllowAny]?

Answer: Because new users registering an account are unauthenticated and do not possess a JWT token yet.

Q116. What is the difference between SpectacularAPIView, SpectacularSwaggerView, and SpectacularRedocView?

Answer: SpectacularAPIView outputs the raw OpenAPI schema (/api/schema/). SpectacularSwaggerView renders interactive Swagger UI (/api/docs/). SpectacularRedocView renders ReDoc (/api/redoc/).

Q117. In docker/prometheus/prometheus.yml, what targets are configured?

Answer: Two scrape targets: django-api targeting web:8000 and celery-workers targeting web:8000, scraped every 5 seconds.

Q118. In docker-compose.yml, what is the purpose of named volumes?

Answer: Named volumes (postgres_data, redis_data, etc.) map container directories to host storage, ensuring databases and dashboards persist across container rebuilds.

Q119. In tests/test_worker_execution.py, why do we mock RedisDistributedLock acquire and release?

Answer: To test state transitions, backoff calculations, and database logging in isolated unit tests without requiring a running Redis instance.

Q120. In tests/test_worker_execution.py, what does @patch('workers.tasks.execute_job_task.apply_async') verify?

Answer: It verifies that when a task fails with retries remaining, apply_async is called once with incremented attempt number and positive countdown delay.

Q121. In config/settings/production.py, what does SECURE_SSL_REDIRECT = True do?

Answer: It redirects all unencrypted HTTP requests to HTTPS via Django's SecurityMiddleware.

Q122. What does SECURE_HSTS_SECONDS = 31536000 do?

Answer: It sets the Strict-Transport-Security header, instructing browsers to strictly use HTTPS for the next 1 year.

Q123. What does SECURE_CONTENT_TYPE_NOSNIFF = True do?

Answer: It sets X-Content-Type-Options: nosniff, preventing browsers from MIME-sniffing responses away from declared Content-Types.

Q124. In config/settings/base.py, what is CORS_ALLOW_ALL_ORIGINS?

Answer: In local.py, it is set to True to allow frontend apps on different ports (e.g. React on 3000) to communicate with API on 8000 without CORS blocking.

Q125. What is the difference between CELERY_TASK_ALWAYS_EAGER = True and regular asynchronous Celery?

Answer: CELERY_TASK_ALWAYS_EAGER=True runs tasks synchronously in the local thread immediately, bypassing Redis so unit tests can evaluate task results instantly.

Q126. What does CELERY_TASK_EAGER_PROPAGATES = True do?

Answer: When eager mode is active, it ensures any exceptions raised inside the task propagate to the test caller.

Q127. In manage.py, why is sys.path.append(str(base_dir / 'apps')) added?

Answer: To allow clean imports like from users.models import User without prefixing every import with apps..

Q128. How does django.utils.timezone.now differ from datetime.datetime.now()?

Answer: timezone.now() is timezone-aware and respects TIME_ZONE = 'UTC', preventing timezone offset bugs across distributed servers.

Q129. What is the purpose of Job.is_terminal property in apps/jobs/models.py?

Answer: It is a boolean helper returning True if status in {COMPLETED, FAILED, CANCELLED}, guarding against invalid state modifications.

Q130. In apps/jobs/views.py, how does cancel() handle an already COMPLETED job?

Answer: It returns HTTP 400 Bad Request stating the job is already in a terminal state.

Q131. What is the role of PrometheusBeforeMiddleware and PrometheusAfterMiddleware?

Answer: BeforeMiddleware records request start time. AfterMiddleware calculates latency and increments request counters by status code for /metrics.

Q132. Why must PrometheusBeforeMiddleware be the very first middleware and AfterMiddleware the last?

Answer: To accurately measure the full execution duration of all intermediate middleware (Authentication, CSRF, Routing).

Q133. What is the purpose of tests/__init__.py?

Answer: It marks the tests directory as a Python package so test modules can share fixtures and imports cleanly.

Q134. In pytest.ini, what does --reuse-db do?

Answer: It instructs pytest-django to reuse the test database across runs instead of dropping and recreating it every time, speeding up test execution.

Q135. In requirements/base.txt, why is psycopg2-binary included?

Answer: It is the PostgreSQL database adapter for Python, enabling Django ORM to communicate with PostgreSQL over TCP.

Q136. What is the difference between requirements base.txt, local.txt, and production.txt?

Answer: base.txt contains core runtime dependencies; local.txt adds testing/linting tools; production.txt adds production servers (Gunicorn) and Sentry.

Q137. How does the API entrypoint script ensure PostgreSQL is ready?

Answer: It runs a while ! nc -z $POSTGRES_HOST $POSTGRES_PORT; do sleep 0.5; done loop probing the TCP socket until PostgreSQL accepts connections.

Q138. In docker/entrypoints/worker.sh, what does -n worker@%h do?

Answer: It assigns a unique node name to the Celery worker process using the container hostname (%h).

Q139. In docker/entrypoints/worker.sh, what does --concurrency=4 do?

Answer: It instructs Celery to spawn 4 child worker processes, allowing the container to process 4 tasks concurrently.

Q140. How does Celery handle task execution across multiple worker processes?

Answer: The master Celery process listens on Redis and delegates task messages to child worker processes using an OS IPC pipe.

Q141. What is the difference between Celery's prefork, gevent, and threads execution models?

Answer: prefork uses multiprocessing for CPU-bound tasks; gevent uses greenlets for massive concurrent I/O; threads uses OS threads.

Q142. What pool execution model is used in this project?

Answer: prefork (multiprocessing), providing strong memory isolation between worker tasks.

Q143. In apps/jobs/models.py, what does auto_now=True on updated_at do?

Answer: It automatically updates the field to the current timestamp whenever job.save() is called.

Q144. In apps/jobs/models.py, what does auto_now_add=True on created_at do?

Answer: It sets the field to the current timestamp only once when the instance is first created in the database.

Q145. Why did you use default=timezone.now instead of auto_now_add=True on Job.created_at?

Answer: default=timezone.now allows the field to be manually overridden in test fixtures or backfills, whereas auto_now_add=True forcefully overwrites passed values.

Q146. In apps/users/models.py, why is email marked with db_index=True and unique=True?

Answer: unique=True creates a unique B-tree index guaranteeing uniqueness and optimizing authentication queries to O(log N) time.

Q147. What does AUTH_USER_MODEL = 'users.User' tell Django?

Answer: It instructs Django to replace its default auth.User model with our custom users.User model across all foreign keys and admin interfaces.

Q148. In apps/users/serializers.py, what does RefreshToken.for_user(user) do?

Answer: It generates a signed JWT Refresh Token containing user_id and jti claims, and derives an Access Token from it.

Q149. What is a jti claim in a JWT?

Answer: JWT ID — a unique UUID string assigned to each individual token used by the token blacklist to track and revoke specific tokens.

Q150. In apps/jobs/serializers.py, why is status a read-only field in JobCreateSerializer?

Answer: Clients must not dictate initial status; the server strictly enforces the initial state as QUEUED.

Q151. In apps/jobs/serializers.py, why is id a read-only field in JobCreateSerializer?

Answer: The primary key UUID is generated by the server upon creation, preventing clients from dictating arbitrary primary keys.

Q152. In apps/jobs/serializers.py, what does ChoiceField(choices=JobPriority.choices) enforce?

Answer: It ensures the API accepts only valid priorities ('low', 'medium', 'high'), returning HTTP 400 for invalid strings.

Q153. What is the purpose of apps/workers/apps.py?

Answer: It defines WorkersConfig(AppConfig), enabling Django to discover Celery tasks and signals inside the workers module.

Q154. What is the purpose of config/__init__.py containing from .celery import app as celery_app?

Answer: It ensures the Celery app instance is loaded whenever Django starts so @shared_task decorators bind correctly.

Q155. In apps/jobs/idempotency.py, what does response['X-Cache-Lookup'] = 'HIT-Idempotency' do?

Answer: It injects an HTTP header informing clients that the response was served directly from the idempotency cache without re-executing task creation.

Q156. In apps/jobs/idempotency.py, what does response['X-Cache-Lookup'] = 'MISS-Idempotency-Saved' do?

Answer: It indicates that this was the first time the request was processed and its response was cached for future retries.

Q157. What is the time complexity of looking up an idempotency key in PostgreSQL?

Answer: O(log N) where N is row count, because (user_id, idempotency_key) is backed by a PostgreSQL B-tree index.

Q158. In apps/workers/locking.py, what is the time complexity of acquiring a lock in Redis?

Answer: O(1) constant time, because Redis performs hash table key lookups and inserts in memory.

Q159. In apps/workers/locking.py, what is the time complexity of releasing a lock via Lua script?

Answer: O(1) constant time for single-key evaluation.

Q160. How does your test suite test multi-tenant isolation?

Answer: User A creates a job. User B authenticates and attempts GET /api/jobs/{id}/ and POST cancel. The test asserts response status is 404 Not Found for both.

Q161. Why does User B get 404 Not Found instead of 403 Forbidden when accessing User A's job?

Answer: Returning 403 confirms the resource ID exists. Returning 404 hides resource existence entirely, preventing ID enumeration.

Q162. How does test_job_cancellation_terminal_state_fails verify terminal state protection?

Answer: It sets job status to COMPLETED, calls cancel, and asserts status is 400 with an error stating the job is already terminal.

Q163. In tests/test_auth.py, how do you test token refresh rotation?

Answer: The test logs in, receives a refresh token, calls POST /api/auth/refresh/, and asserts status is 200 with a new access token.

Q164. In tests/test_auth.py, how do you test token blacklisting?

Answer: The test logs out with the refresh token (blacklisting it), then immediately calls /api/auth/refresh/ with the same token, asserting status is 401 Unauthorized.

Q165. What happens if a Celery worker encounters an unhandled Python exception?

Answer: Celery catches the exception, logs a traceback, marks the backend result as FAILURE, and triggers failure handlers. In our code, we explicitly catch Exception to update PostgreSQL.

Q166. In apps/workers/tasks.py, why do you truncate error_traceback to 500 characters in job.error_message?

Answer: job.error_message is for quick dashboard summaries. The full untruncated traceback is preserved in JobExecution.error_traceback.

Q167. How would you handle database connection pooling in high-concurrency production on AWS?

Answer: Use AWS RDS Proxy or PgBouncer in front of PostgreSQL to pool and multiplex 500 worker connections into 20-30 shared database connections.

Q168. What is the difference between Horizontal and Vertical Scaling for this system?

Answer: Vertical Scaling adds CPU/RAM to single Postgres/Redis servers. Horizontal Scaling adds more stateless Django API replicas and more Celery worker containers.

Q169. Can Redis become a single point of failure (SPOF) in this architecture?

Answer: In production on AWS, we use AWS ElastiCache for Redis in Multi-AZ configuration with Automatic Failover, promoting a replica in < 30s if primary fails.

Q170. Can PostgreSQL become a single point of failure?

Answer: In production, we deploy AWS RDS PostgreSQL Multi-AZ with synchronous physical replication maintaining a hot standby in another AZ.

Q171. What is the difference between synchronous and asynchronous database replication?

Answer: Synchronous writes to primary and standby before confirming commit (zero data loss, higher latency). Asynchronous confirms primary first, then syncs (lower latency, minimal data loss risk on sudden crash).

Q172. What is a Dead-Letter Queue (DLQ)?

Answer: A secondary queue where messages that repeatedly fail after maximum retries are moved for manual inspection, preventing poison pills from blocking healthy queues.

Q173. How would you migrate this architecture from Redis to RabbitMQ?

Answer: Change CELERY_BROKER_URL to amqp://user:pass@rabbitmq:5672/. Kombu handles AMQP protocol natively, providing native AMQP priority queues.

Q174. Why is Redis better than Kafka for simple priority task queues?

Answer: Kafka is an append-only log optimized for sequential streaming; it does not support arbitrary priority reordering. Redis lists/ZSETs allow true dynamic priority popping.

Q175. What is the difference between CPU-bound and I/O-bound tasks in our worker pool?

Answer: I/O-bound tasks spend time waiting on network/disk (benefits from high concurrency threads). CPU-bound tasks saturate CPU cores (requires multiprocessing bounded by physical cores).

Q176. In apps/jobs/models.py, why does JobResult use models.OneToOneField?

Answer: OneToOneField creates a UNIQUE constraint on job_id, mathematically enforcing that a job can produce at most one final result record.

Q177. In apps/jobs/models.py, what does on_delete=models.CASCADE on Job.user do?

Answer: If a user is deleted, all associated Job, JobResult, JobExecution, and IdempotencyKey records owned by that user are automatically deleted by PostgreSQL.

Q178. What does on_delete=models.SET_NULL on IdempotencyKey.job do?

Answer: If a Job is deleted, the referencing IdempotencyKey row is retained for audit, but its job_id foreign key is set to NULL.

Q179. In apps/users/models.py, what does is_staff vs is_superuser mean?

Answer: is_staff controls access to Django Admin (/admin/). is_superuser grants all model permissions implicitly without specific group assignment.

Q180. In config/urls.py, what does path('admin/', admin.site.urls) do?

Answer: It mounts Django's built-in administrative management portal at /admin/.

Q181. In apps/jobs/admin.py, what is JobExecutionInline?

Answer: A TabularInline class rendering the complete history of JobExecution attempt rows directly inside the parent Job admin page.

Q182. In apps/jobs/admin.py, why are fields in JobExecutionInline marked readonly?

Answer: Execution logs are immutable historical audit records; administrators must not tamper with execution durations or tracebacks.

Q183. In apps/jobs/admin.py, why is can_delete = False set on inlines?

Answer: To prevent administrators from accidentally deleting individual audit execution attempts from the admin UI.

Q184. In apps/monitoring/views.py, how does HealthCheckView check database health?

Answer: It executes cursor.execute('SELECT 1;'). If PostgreSQL answers, it returns 1 and sets status: healthy. If an exception occurs, it returns HTTP 503.

Q185. In apps/monitoring/views.py, how does HealthCheckView check Redis health?

Answer: It runs django_redis.get_redis_connection('default').ping(). Redis returns PONG (True) on success.

Q186. In apps/monitoring/views.py, what metrics are returned in the health check JSON?

Answer: Real-time counts of queued_jobs, processing_jobs, failed_jobs, and completed_jobs queried from PostgreSQL.

Q187. In docker-compose.yml, what does restart: unless-stopped do?

Answer: It instructs Docker to restart containers automatically if they crash or host reboots, unless explicitly stopped by an administrator.

Q188. In Dockerfile, what does ENV PYTHONUNBUFFERED=1 do?

Answer: It forces Python standard output and error streams to send unbuffered logs directly to terminal/Docker logs in real time.

Q189. In Dockerfile, what does ENV PYTHONDONTWRITEBYTECODE=1 do?

Answer: It prevents Python from writing .pyc files to disk inside the container, saving disk space and avoiding stale bytecode issues.

Q190. In Dockerfile, why do we run rm -rf /var/lib/apt/lists/* after apt-get install?

Answer: To delete package index files downloaded during apt-get update, reducing Docker image size by 30-50MB.

Q191. In docker-compose.yml, why is Prometheus configured with :ro on prometheus.yml?

Answer: It mounts configuration as read-only, preventing container processes from modifying host configuration files.

Q192. How does Prometheus handle target downtime?

Answer: If /metrics is unreachable, Prometheus marks target state as DOWN (up == 0) and fires alerts if configured.

Q193. In apps/monitoring/metrics.py, why are specific buckets defined on JOB_EXECUTION_SECONDS?

Answer: Buckets (0.1s to 120s) define latency histogram bins for tracking task distribution from fast 100ms tasks to 2-minute heavy tasks.

Q194. In apps/workers/tasks.py, why is ACTIVE_JOBS_GAUGE.dec() inside a finally block?

Answer: Placing decrement in finally guarantees active job count is decremented whenever the worker exits, regardless of success, failure, or cancellation.

Q195. In tests/load_testing/locustfile.py, what does @task(5) vs @task(1) mean?

Answer: It sets relative execution weight; task(5) is chosen 5 times more frequently by simulated users than task(1).

Q196. What is the difference between Latency and Throughput?

Answer: Latency is time taken to complete a single unit of work (ms/seconds). Throughput is number of units completed per unit of time (requests/sec or jobs/min).

Q197. What is p99 latency?

Answer: 99th percentile latency — 99% of requests/jobs finished faster than this threshold, measuring worst-case tail performance.

Q198. What are the key limitations of this project in its current state?

Answer: Single-node Redis broker, polling-based client status checks (instead of WebSockets/SSE), and monolithic repository layout.

Q199. What are the planned future extensions for this platform?

Answer: WebSocket/SSE real-time push updates, dynamic task DAG workflows (Chains/Chords), and multi-region disaster recovery.

Q200. Summarize why this project proves readiness for a Backend / SDE role.

Answer: It demonstrates mastery across ACID relational database design, message queue topologies, distributed locking, jittered backoff recovery, idempotency, API security, automated testing, container orchestration, and telemetry observability.